
KeyForge

Mechanical Keyboard E-Commerce Platform

A Full-Stack Web Application — React · FastAPI · MongoDB

Final Project Presentation

Members : 1 Kimhong Pech

2 Sophal Pheakdeypanhasak

3 Sreang Limsry

4 Chea Lymeng

5 Pheak Solyna

6 Kongrith Vireaksal

Live: <https://key-forge-final-project.vercel.app> **Repo:** github.com/kimhongpech-dev/KeyForge-Final-Project

OUTLINE

- 01 Project Overview & Goals**
What KeyForge is, who it serves, and what it achieves
- 02 Problem & Motivation**
Why build a full e-commerce platform from scratch
- 03 Technology Stack**
Frontend, backend, database, and deployment choices
- 04 System Architecture**
How the pieces talk to each other
- 05 Database & API Design**
MongoDB data model and REST endpoints
- 06 Core Features**
Catalog, auth, cart & checkout, orders, admin dashboard
- 07 UX & Design**
Theming, responsiveness, accessibility
- 08 Deployment & Challenges**
GitHub, Vercel, MongoDB Atlas, problems solved
- 09 Testing, Demo & Future Work**
How it was verified, demo flow, next steps

Project Overview

KeyForge is a production-style online store for mechanical keyboards, gaming mice, and earphones — built end-to-end, from product browsing to admin analytics.

Key Goals

- ▶ **Complete shopping journey:** browse → search → cart → checkout → track orders
- ▶ **Real accounts & security:** registration, login, JWT sessions, admin roles
- ▶ **Business-grade stock control:** inventory enforced atomically at checkout
- ▶ **Full admin operations:** product CRUD, stock edits, order workflow, analytics
- ▶ **Production deployment:** live on Vercel with MongoDB Atlas, auto-deploy on push

6

Charts & stats

custom-built SVG analytics

2

Theme modes

light & dark, remembers choice

1

Owner / admin account

boss@keyforge.com · promoted via script

Deployed live at key-forge-final-project.vercel.app

Problem & Motivation

A shopping site is a small operating system: products, money, identity, and inventory must all stay consistent in real time. Most course projects stop at mock data — KeyForge goes the full distance to a working, deployed product.

The Problem

Fake checkouts

orders that never reduce stock or create records

No real accounts

everyone sees everything, no roles or ownership

Manual operations

no way to manage products, stock, or orders

Static catalog

products hard-coded in the frontend, unchangeable

No deployment

works on one laptop only, impossible to share

How KeyForge Solves It

Real checkout

orders persisted, stock decremented atomically

JWT accounts & roles

customers vs. admin, protected APIs & pages

Admin dashboard

full product CRUD, stock, order workflow, analytics

Database-driven catalog

25 products served from MongoDB, editable

Production deployment

live on Vercel, MongoDB Atlas, CI via git push

Technology Stack

Frontend

React 19 + Vite

fast builds, HMR, component model



React Router

7 pages, dynamic product routes



Context API

auth, cart & theme state — no extra libraries



Custom CSS

design-system variables, responsive grid



Backend

Python 3.12 + FastAPI

async, typed, auto-documented



Motor (async driver)

non-blocking MongoDB access

PyJWT + bcrypt

signed sessions, hashed passwords

Pydantic schemas

validated request/response models

Data & Ops

MongoDB Atlas

cloud database, free tier, auto-indexed



GitHub

private repo, versioned history



Vercel

static site + Python serverless functions



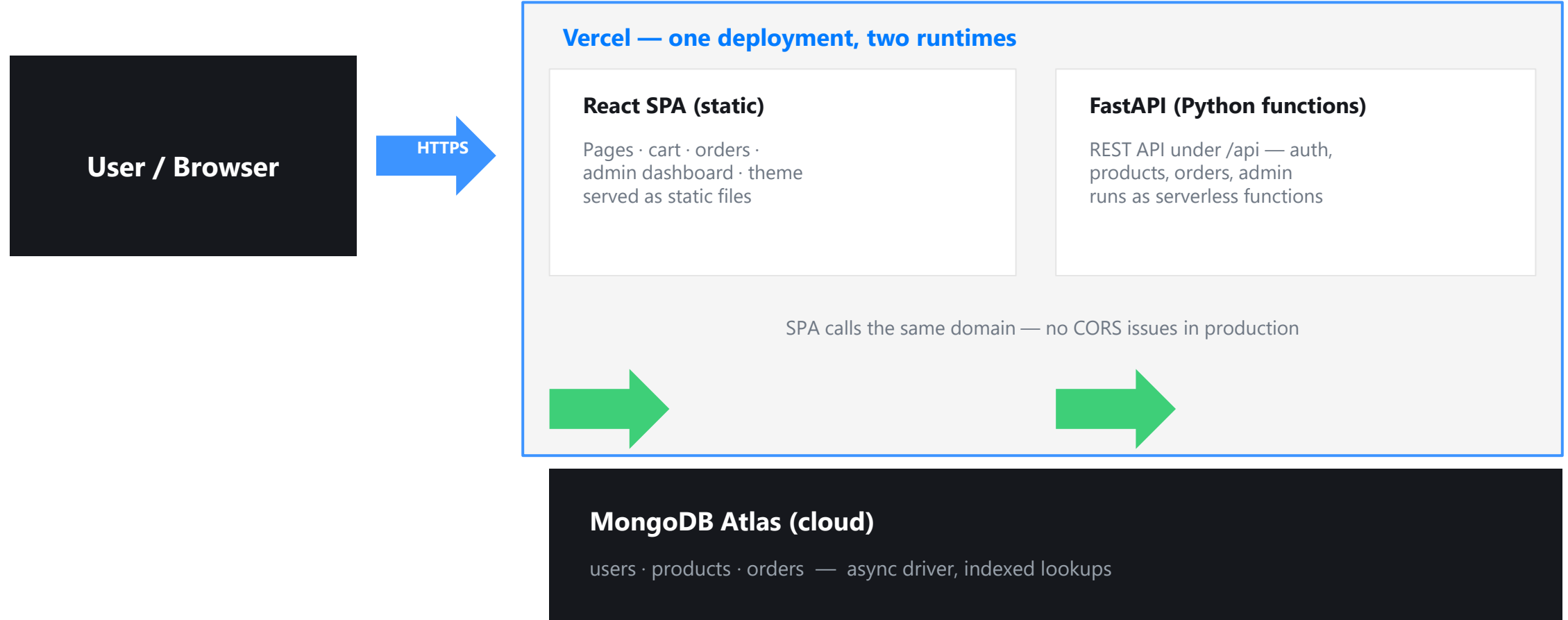
GitHub ↔ Vercel

auto-deploy on every push to main



No heavy UI libraries — charts, theming, and layout are hand-built for full control and learning value.

System Architecture



Key flows: JWT issued at login and verified per request · cart lives in browser state · orders & stock updates are atomic database operations.

Database Design — MongoDB Atlas

Three collections, one database. Order items are embedded (no join needed); stock lives on the product so it can be decremented atomically.

users

name, email (unique)
password — bcrypt hash
is_admin flag
created_at

products

id, name, price
category, description, image
stock — source of truth
25 seeded records

orders

user email + items embedded
total, status, timestamps
tracking: 4 status values

Entity-relationship overview (from the project's design phase):



REST API Design

Method	Endpoint	Purpose
GET	/api/products	List products · optional ?search & ?category filters
GET	/api/products/{id}	Single product detail
GET	/api/products/categories	Distinct category list for filters
POST	/api/auth/register	Create account (email unique, password hashed)
POST	/api/auth/login	Verify credentials, return JWT token
GET	/api/auth/me	Resolve current user from token
POST	/api/orders	Place order · atomic stock decrement · rejects short stock
GET	/api/orders	List my orders (newest first)
POST	/api/orders/{id}/cancel	Client cancellation · restores stock
GET	/api/admin/products	Admin product list
POST / PUT / DELETE	/api/admin/products/{id}	Admin product CRUD & stock updates
GET / PUT	/api/admin/orders/{id}	Admin order list & status workflow
GET	/api/admin/stats	Dashboard analytics (revenue, statuses, categories, top)

All errors return a consistent shape — `{"error": "<message>"}` — handled by one global exception handler.

Feature — Product Catalog & Discovery

What the customer sees

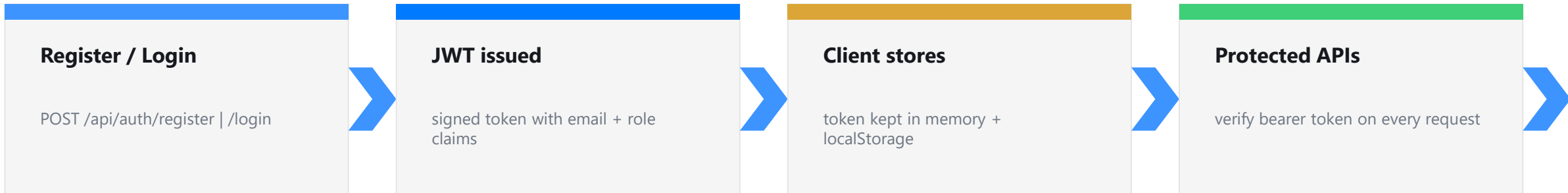
- ▶ **Live search**
matches product name & description on the server
- ▶ **Category filters**
Keyboard · Mouse · Earsphone — one click filtering
- ▶ **Stock awareness**
cards show live stock; out-of-stock items can't be added
- ▶ **Product details page**
full description, price, and add-to-cart with stock caps

How it's implemented

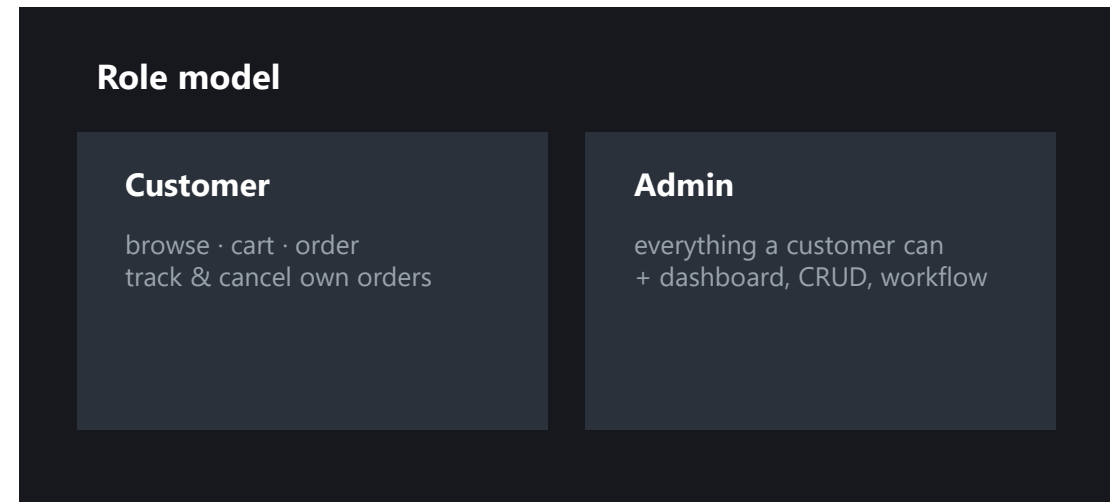
- ▶ **Server-side filtering:** FastAPI builds a MongoDB query from query params
- ▶ **Cache in memory:** first fetch fills a module-level cache per session
- ▶ **Reusable components:** ProductCard, SearchBar, CategoryFilter
- ▶ **Clean state:** one products[] state driven by search / category effects

Design detail: the API returns JSON with stock included, so every card always shows the current inventory — no stale numbers after an order.

Feature — Authentication & Authorization



- ▶ **Passwords never stored in plain text** bcrypt hashing with per-user salt
- ▶ **Stateless sessions** PyJWT signs the token; no server session storage
- ▶ **Admin flag in token** the API trusts role claims, the UI hides admin links
- ▶ **Edge cases handled** duplicate email → 400, wrong password → 401, expired token → re-login



Demo account: boss@keyforge.com (admin)

Feature — Cart & Checkout

Customer experience

- ▶ **Global cart state**

Context provider so every page shares one cart

- ▶ **Quantity controls**

add, remove, and adjust — capped at available stock

- ▶ **Live totals**

subtotal recalculated from cart items automatically

- ▶ **Confirmation screen**

order number, summary, and items after success

Stock safety — the interesting part

1. Checkout reads stock from the database (not the browser)
2. Order placed in one atomic update: `{"$inc": {"stock": -qty}}`
3. If any item is short, the whole order is rejected — **no partial decrements**
4. Client cart is cleared only on success

Why this matters

Two customers hitting the last unit at the same moment: MongoDB's atomic update means exactly one succeeds. The loser gets a clear "insufficient stock" error with the item name — no negative inventory, no double sales.

Feature — Order Tracking & Cancellation



Feature — Admin Dashboard & Analytics

Operations

- ▶ **KPI stat cards**revenue, order count, stock-outs, low stock — from one /stats call
- ▶ **Product management**create, edit, delete, and update stock in place
- ▶ **Order workflow**move any order pending → confirmed → shipped → delivered
- ▶ **Search & filter**find products and orders instantly
- ▶ **Role-gated**route + API both reject non-admins

Analytics — hand-built SVG charts

- ▶ **Revenue last 14 days**bar chart with value labels & gridlines
- ▶ **Orders by status**color-coded bars mirroring the workflow colors
- ▶ **Products by category**donut with legend, counts and percentages
- ▶ **Top products**horizontal bars by quantity sold + revenue
- ▶ **No chart library**pure SVG components — precise control, zero dependencies

API: GET /api/admin/stats aggregates orders and products server-side in one round trip.

UX & Visual Design

Theming

- ▶ **Light & dark themes**
one toggle in the navbar, saved in localStorage
- ▶ **Follows the OS by default**
prefers-color-scheme on first visit
- ▶ **No flash on load**
inline script sets the theme before React mounts
- ▶ **Design tokens**
196 colors → 38 semantic CSS variables

Craft details

- ▶ **Responsive layout**desktop grid, mobile hamburger menu, fluid type
- ▶ **Accessibility**aria labels, keyboard focus rings, semantic buttons
- ▶ **State coverage**loading, empty, error, out-of-stock, success — all styled
- ▶ **Consistent feedback**one error shape across every API failure

The full palette is token-driven

Themes swap only the token values (`--bg`, `--surface`, `--text...`) — components never change, and dark-mode contrast stays $\geq 4.5:1$ for body text.

Deployment & DevOps

The whole stack runs on one Vercel project: the React build is served as static files, and the same vercel.json routes /api/* to a Python 3.12 serverless function. MongoDB runs on Atlas — zero servers to manage.

Source control — GitHub

- private repo, main branch
- clear commit history
- identity configured for the owner

Hosting — Vercel

- static build (Vite → dist)
- Python functions via vercel.json
- env vars: MONGODB_URI, JWT_SECRET
- auto-deploy on every git push

Database — MongoDB Atlas

- free cloud cluster
- network access for the server
- seeded once via script

Deploy pipeline: git commit → push to GitHub main → Vercel builds the site + bundles the API → production alias updates in ~1 minute.

Challenges & How We Were Solved

Products disappeared under category filters

Problem: Three seeded products had no category field, so their category was empty and the filter matched nothing.

Fix: Added the missing categories in the seed data and re-seeded the database.

Product images broken in production

Problem: 22 of 25 images pointed at /src/assets paths that only exist on the dev machine, not on Vercel.

Fix: Mounted the assets directory in FastAPI and routed /src/assets/* to the backend in vercel.json.

Vercel blocked every new deployment

Problem: Hobby plan rejects deployments whose git commit author isn't the account owner; CLI deploys used a placeholder identity.

Fix: Rewrote the commits under the owner's real identity and connected the GitHub repo for auto-deploys.

Lesson: data hygiene, environment parity, and identity configuration are the three silent killers of a demo day.

Testing & Verification

Build pipeline

- npm run build passes cleanly (Vite production build)
- eslint runs with only pre-existing warnings
- Vercel build + Python bundle succeed

Edge cases covered

- duplicate signup email → friendly 400
- out-of-stock products can't be added
- admin pages blocked for non-admins
- dark/light theme persists across reloads

All checks are repeatable: the same commands (build, lint, API calls) are documented in the repo and were re-run after every fix.

Running the Project Locally

Three commands

npm install

install frontend dependencies

npm run seed

load 25 products into MongoDB

npm run dev:all

Vite (port 5173) + FastAPI (port 5000) together

A .env file holds MONGODB_URI and JWT_SECRET (never committed).

Project structure

src/

React pages, components, contexts, data layer

server/

FastAPI app: routers, schemas, security, seeding

api/index.py

Vercel entry point — re-exports the FastAPI app

vercel.json

builds & routes: static + Python on one domain

requirements.txt

backend dependencies (both local & Vercel)

Future Improvements

Payments

Integrate Stripe or PayPal for real checkout instead of order placement

Wishlists & reviews

per-user favorites and product ratings

Email notifications

order confirmation and status change emails

Shipping

address book, courier selection, live tracking numbers

Inventory alerts

low-stock notifications to the admin

Performance & scale

server-side pagination, image CDN, response caching

The architecture is already shaped for these: every feature is a new route, a new context, or a new collection — no rewrites.



Thank You

Questions & feedback are welcome

Live demo: <https://key-forge-final-project.vercel.app>

Source: <https://github.com/kimhongpech-dev/KeyForge-Final-Project>